

人工智能原理与应用

詹紫琳

邮箱: zilin.zhan@szu.edu.cn

Spring 2026



深圳大学
SHENZHEN UNIVERSITY

回顾

理性地行动 (Acting Rationally): 在环境中，为达成**目标**而行动

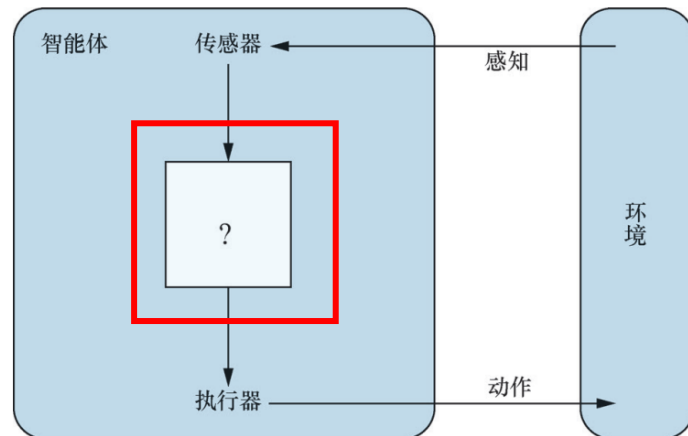
	经济学 (Economics)	人工智能 (Artificial Intelligence)
研究主体	理性人 (Homo Economicus) 拥有明确的偏好 (Preferences) 面临外部的约束 (Constraints)	智能体 (Agent) 拥有可度量的目标 (Goal/Performance Measure) 身处于特定的环境 (Environment)
基本假设	基于此做出最优选择 (Choices)	通过感知与行动做出决策 (Decisions via Perception & Action)

3/ 搜索求解

回顾

智能体是怎么思考的？

- CH 3-4 问题求解（单智能体、完全可观测、确定性、静态）
——应用：八数码问题、迷宫导航、棋盘谜题
- CH 7-12 知识、推理、规划（单智能体、完全可观测、确定性、动态）
——应用：机器人组装任务、物流调度
- CH 13-17 不确定知识与推理（单/多智能体、部分可观测、不确定性）
——应用：自动驾驶在不确定环境下的决策、医疗诊断
- CH 18-21 学习（智能体提升性能的横向机制，适用于所有环境）
——应用：AlphaGo（深度强化学习）、广告推荐系统

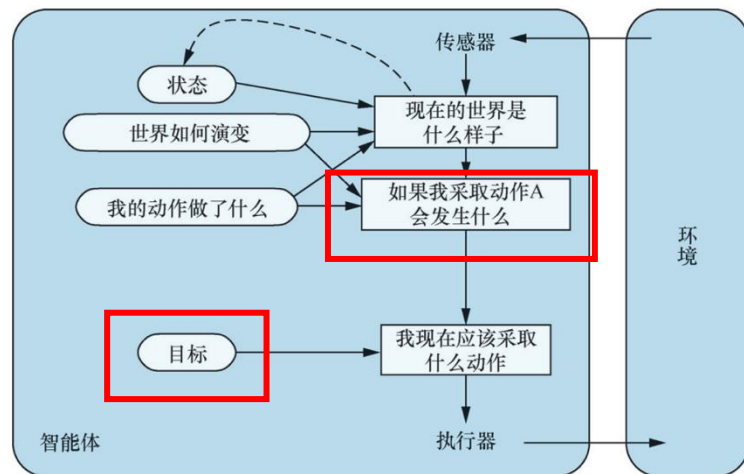


回顾

基于目标的智能体

工作原理：

- 不仅知道世界是怎样的 (**initial state**) 还知道自己**想达到什么状态** (**goal state**)。
- 行动选择基于“哪个行动能让我最接近**目标**?”。
(依靠-搜索 (Search) 和 规划 (Planning))



在确定目标后，如何实现目标？

——**问题求解** (problem-solving) /规划!

问题求解

- 定义：在**当前状态 (initial state)** 与 **目标状态 (goal state)** 之间存在差距时，个体或系统通过一系列**推理、搜索和行动**来弥合这一差距的过程
- 有明确目标状态，只关心**是否到达目标**（不考虑目标合理与否、方法优劣）



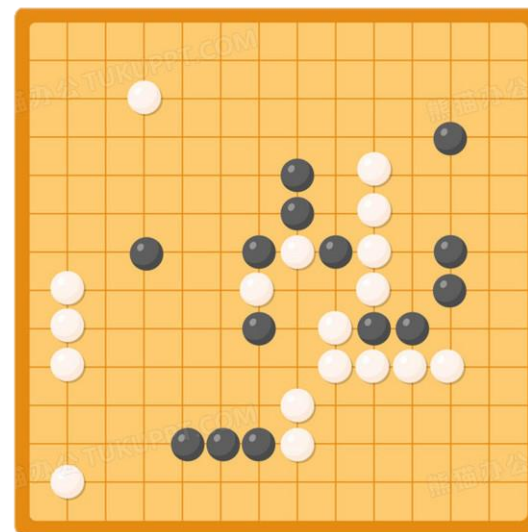
- e.g., 迷宫里当前的位置坐标(3,4)
- e.g., 迷宫里走到出口 (0,0)

状态 (State) 表示的三种方法

- 每个状态就是一张“照片”（不可分割的整体）
- 表示形式: *State 1*, *State 2*, *State 3*....
- E.g., 棋盘上的棋谱摆法

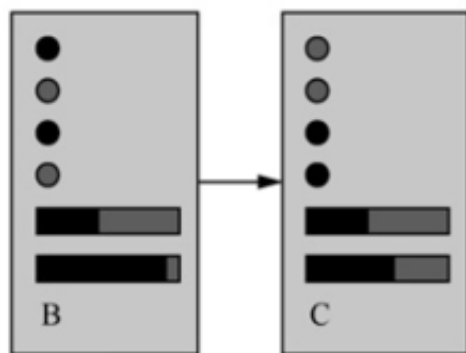


(a) 原子表示



状态 (State) 表示的三种方法

- 由多个属性/变量组成，每个变量都有取值
- 一张“照片”被拆开成多个“元素标签”，比如人物、地点、时间等
- 表示形式: $State1 = \{Variable\ 1=Value\ 1, Variable\ 2=Value\ 2, Variable\ 3=Value\ 3 \dots\}$

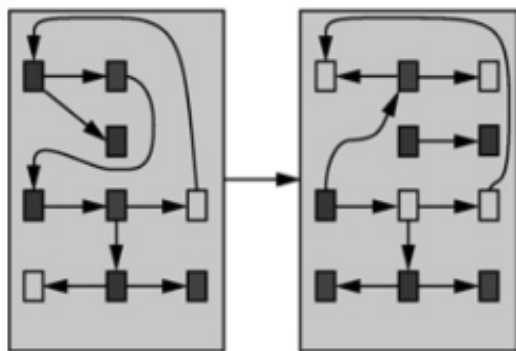


(b) 因子化表示

E.g., 围棋棋盘的状态用每个交叉点的属性来表示: $(3,3) = \text{黑}$, $(4,4) = \text{白}$, $(10,10) = \text{空} \dots$

状态 (State) 表示的三种方法

- 元素+元素间的关系网络
- 表示形式: $状态 = \{对象 + 对象之间的关系\}$
- E.g., 围棋棋盘的状态可以用 棋子与棋子的连接关系表示:
 - 黑子A 与 黑子B 相邻, 形成一个“气”;
 - 白子C 被黑子群团包围;
 - 黑子群D 还有 2 个气。



(c) 结构化表示

问题求解

关键 —— 精确定义问题和解！

- **问题表示 (Problem Formulation)**：如果一个问题找不到一个合适的表示方法，就谈不上对它求解。
- **求解方法**：由于绝大多数需要人工智能方法求解的问题缺乏直接求解的方法，因此，**搜索 (Searching)** 为一种求解问题的一般方法

为什么要学习搜索算法？

- 人工智能最早期的方法
- **不依赖**大规模数据学习，只依赖显式规则和启发函数
- **搜索算法是人工智能的元方法**：无论是机器学习、强化学习还是大模型，本质上都在面对一个庞大**状态或参数空间**的搜索问题

搜索

无信息搜索（Uninformed Search）

不利用问题特定领域知识，只依赖问题定义。

- 广度优先搜索、深度优先搜索、一致代价搜索、深度限制搜索、迭代加深搜索

有信息搜索（Informed Search）

使用启发式函数（领域知识）来估计节点到目标的代价或距离

- 贪婪最佳优先搜索、A*搜索、启发式搜索

局部搜索

对抗性搜索

搜索 Step 1-问题定义 (Problem Formulation)

目标：把现实世界的问题抽象成 AI 可以处理的形式

定义一个搜索问题，需要以下因素

1. 要搜索的**状态空间 (State Space)**。状态空间必然涉及到对实际问题的抽象。
2. 最能代表你当前状态的**初始状态 (Initial Space)**。
3. 你想要达到的**目标状态 (Goal State)**。
4. 允许在状态间移动的**行动 (或后续函数) (Successor Function)**
(给定一个状态 s 对应所有可能的 (动作, 新状态) 对)

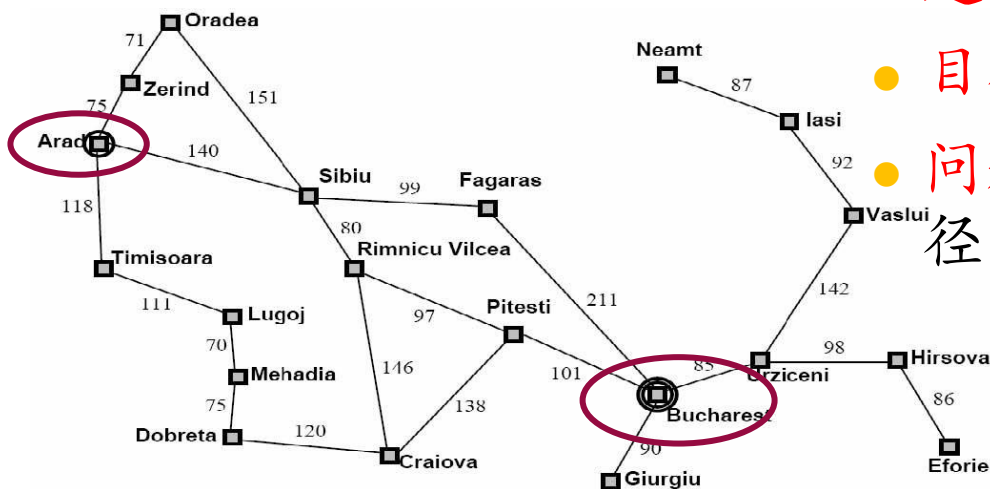
可选项

- **成本/路径耗散 (Path Cost)**，代表从一个状态到另一个状态（采取行动，前进到下一个状态）的成本。
- 启发规则，帮助指导搜索过程。

搜索 Step 1-问题定义 (Problem Formulation)

罗马旅行问题：怎么从Arad到Bucharest?

- **状态空间**：罗马地图上所有城市
- **动作集**：从当前城市出发，沿着道路前往某个邻接城市
- **起始状态**：在Arad
- **目标状态**：到达Bucharest
- **问题求解**：一条城市序列（路径），前往Bucharest的城市顺序



搜索 Step 1-问题定义 (Problem Formulation)

八数码问题

7	2	4
5		6
8	3	1

起始态

1	2	3
4	5	6
7	8	

终止态

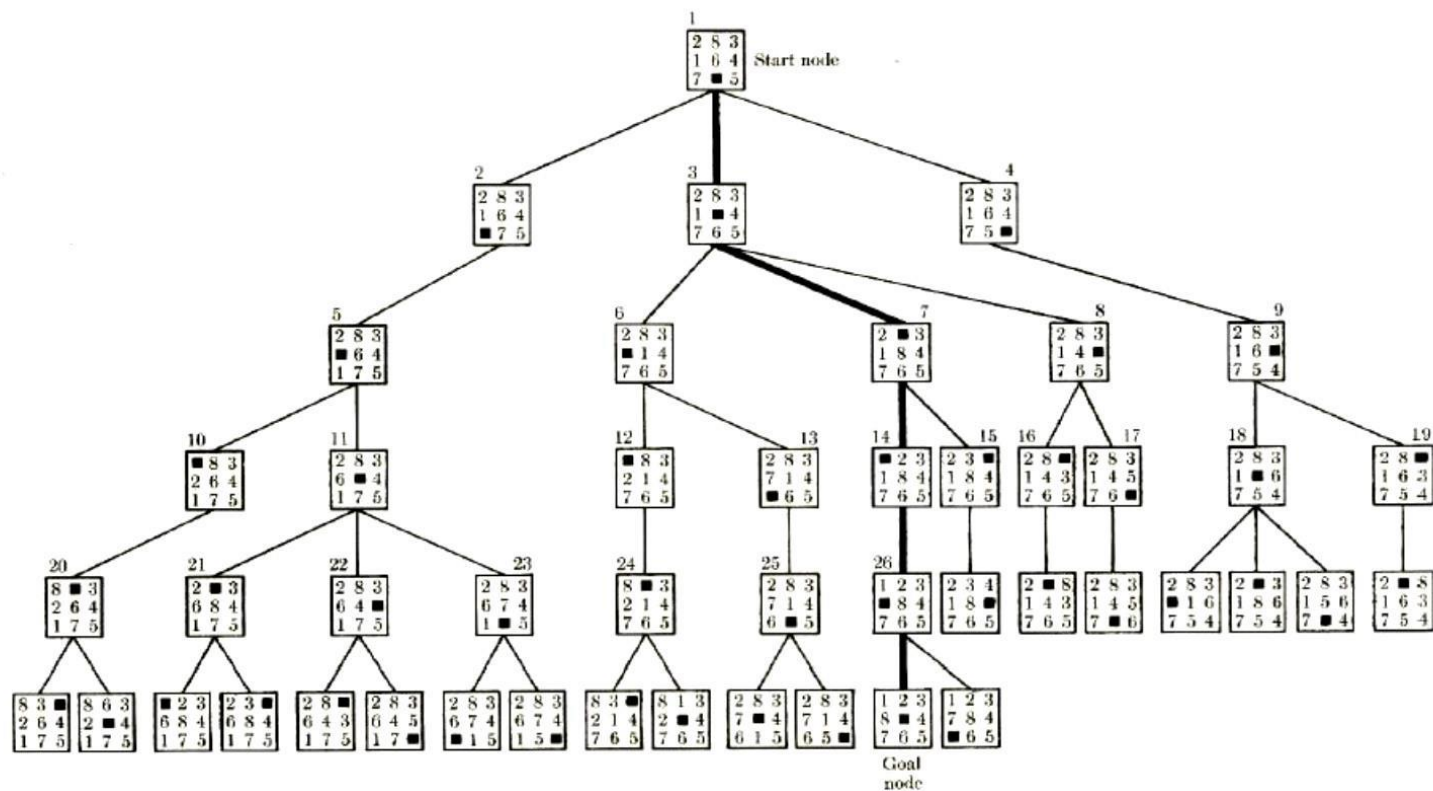
- 状态空间
- 动作集
- 起始状态
- 目标状态
- 问题求解

规则：可以将方片滑入空白点。（等价地，可以把它看作是移动空白）。

(<https://www.mathsisfun.com/games/arrange.html>)

搜索 Step 1-问题定义 (Problem Formulation)

搜索空间：搜索算法在求解过程中，基于状态空间实际展开的部分



搜索 Step 1-问题定义 (Problem Formulation)

水罐问题

- 我们有一个3L的罐子和一个4L的罐子。我们可以用水龙头把其中一个罐子装满，也可以清空其中一个罐子，或者我们可以把一个罐子倒进另一个罐子里（至少直到另一个罐子装满为止）。

搜索 Step 1-问题定义 (Problem Formulation)

- **状态空间:** 数字对 (gal3, gal4), 其中 gal3 是 3 加仑罐子中的加仑数, gal4 是 4 加仑罐子中的加仑数。
- **动作集:** 清空 3 加仑罐子, 清空 4 加仑罐子, 加满 3 加仑罐子, 加满 4 加仑罐子, 倒 3 到 4, 倒 4 到 3。
- **起始态:** 各种, 例如 (0,0)
- **目标状态:** 各种, 例如, (0,2) 或 (*, 3), 其中 * 表示我们不在乎

搜索 Step 1-问题定义 (Problem Formulation)

再回忆一下...

目标：把现实世界的问题抽象成 AI 可以处理的形式

定义一个搜索问题，需要以下因素

1. 要搜索的**状态空间 (State Space)**。状态空间必然涉及到对实际问题的抽象。
2. 最能代表你当前状态的**初始状态 (Initial State)**。
3. 你想要达到的**目标状态 (Goal State)**。
4. 允许在状态间移动的**行动 (或后续函数) (Successor Function)**
(给定一个状态 s 对应所有可能的 (动作, 新状态) 对)

可选项

- **成本/路径耗散 (Path Cost)**，代表从一个状态到另一个状态（采取行动，前进到下一个状态）的成本。
- 启发规则，帮助指导搜索过程。

搜索 Step 2-问题求解 (Problem Solving)

目标：给定一个已经表示好的问题，用**形式化**符号明确**搜索算法的输入/输出**，从而给搜索算法提供精确的输入

输入

- Agent的初始状态/起始态 (Initial State)
- 下一状态函数/后继函数 (Successor Function)
- 目标测试/可达测试 (Goal test) **测试状态是否能达到目标态**
- 单步花费函数/路径耗散 (Path Cost)

输出

- 一条从起始状态到目标状态的路径 (解,solution)

搜索 Step 2-问题求解 (Problem Solving)

- 后继函数 $S(x)$ 产生一个后续状态的集合，后续状态都是在 x 上执行某个动作而产生的
- 我们可以通过**用于获取这些状态的动作**来注释这些状态

$$S(x) = \{ \langle y, a \rangle \mid y \text{ 是在状态 } x \text{ 上执行动作 } a \text{ 得到的} \}$$

- 不同的动作产生不同的后续状态：

$$S(x) = \{ \langle y, a \rangle, \langle z, b \rangle \} \quad (\text{八数码 - 上移和下移得到的布局不同})$$

$$S(x) = \{ \langle y, a \rangle, \langle y, b \rangle \} \quad (\text{多条边通向同一个终点})$$

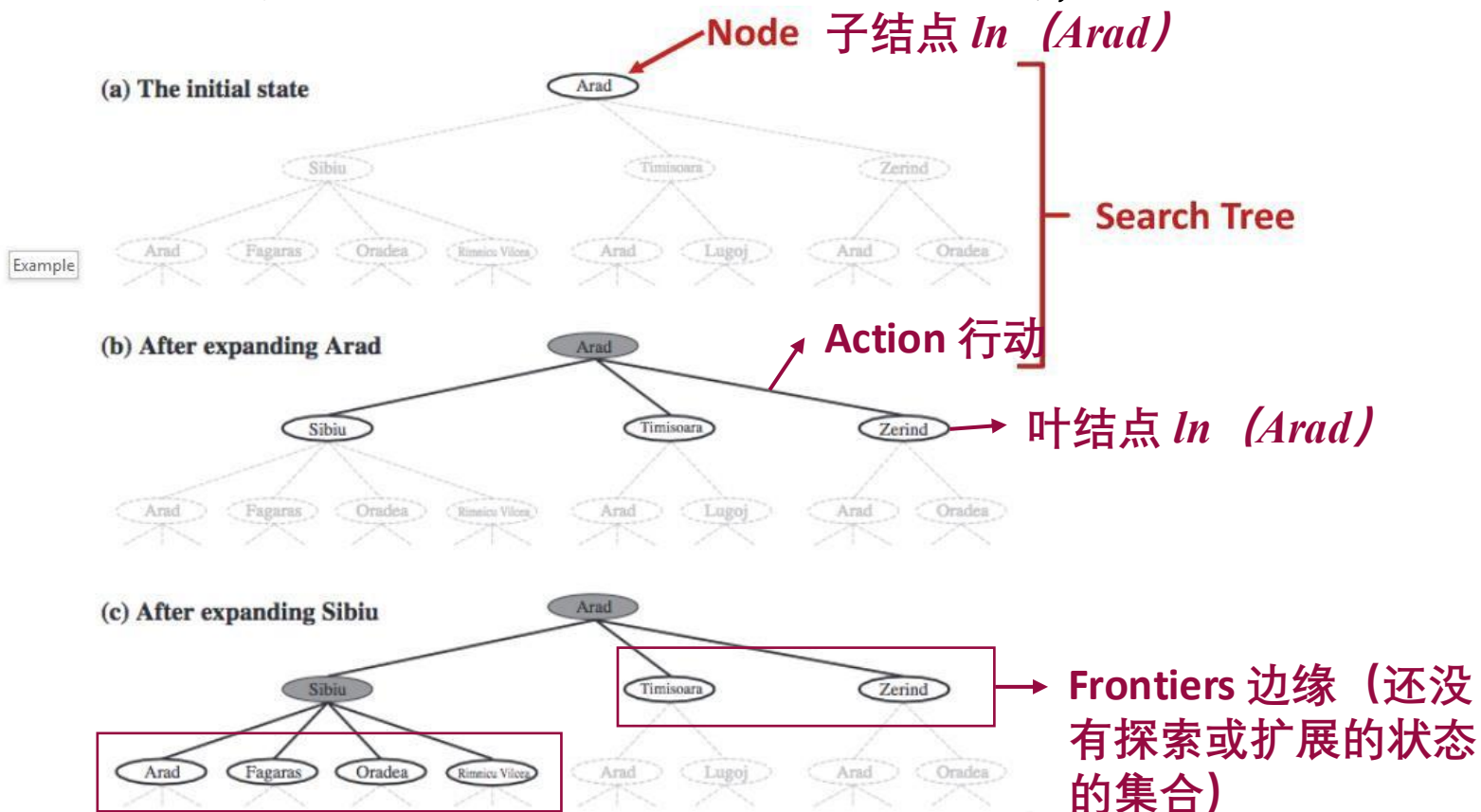
- 如果直接把 x 的所有后续状态都列出来，而不标注动作

$$S(x) = \{y, z\}$$

则丢失了“从 x 到达 y 或 z 是通过哪个动作”的信息，而在搜索过程中，我们不仅需要知道下一个状态，还需要**动作序列**来构成完整解答

搜索求解-盲目搜索(树搜索)

问题求解的输出：一条从起始状态到目标状态的路径（解，Solution）



在边缘中选择结点并扩展，直至找到解或者没有状态可扩展....

搜索求解-盲目搜索(树搜索)

{Arad},

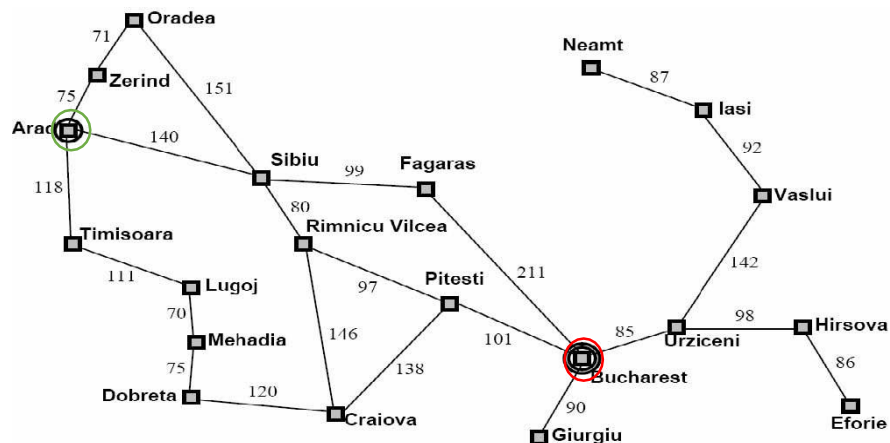
{Z<A>, T<A>, S<A>},

{Z<A>, T<A>, A<S;A>, O<S;A>, F<S;A>, R<S;A>}

{Z<A>, T<A>, A<S;A>, O<S;A>, R<S;A>, S<F;S;A>, B<F;S;A>}

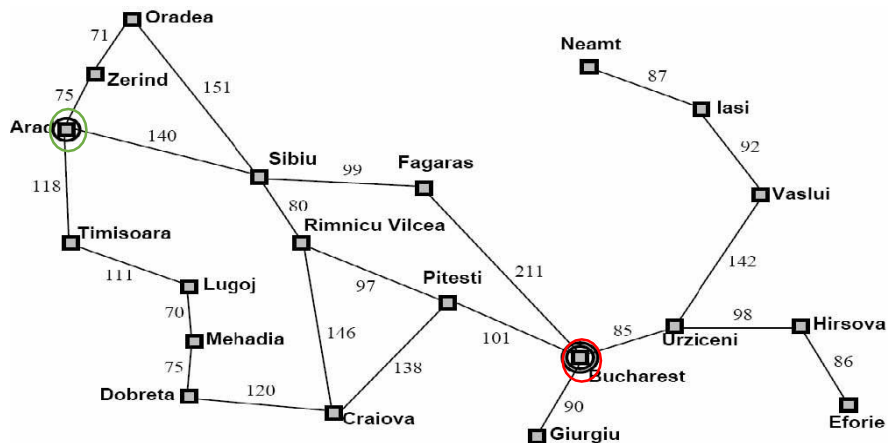
解决方案/Solution: Arad -> Sibiu -> Fagaras -> Bucharest

花费/Path Cost : 140 + 99 + 211 = 450



搜索求解-盲目搜索(树搜索)

不同的解!



{Arad},

{Z<A>, T<A>, S<A>},

{Z<A>, T<A>, A<S,A>, O<S,A>, F<S,A>, R<S,A>}

{Z<A>, T<A>, A<S,A>, O<S,A>, F<S,A>, S<R,S,A>, P<R,S,A>, C<R,S,A>}

{Z<A>, T<A>, A<S,A>, O<S,A>, F<S,A>, S<R,S,A>, C<R,S,A>, R<P,R,S,A>, C<P,R,S,A>, Bucharest<P,R,S,A>}

解决方案: Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest

花费: 140 + 80 + 97 + 101 = 418

搜索求解-盲目搜索(树搜索)

function TREE-SEARCH(*problem*) **returns** a solution, or failure

initialize the **frontier** **using** the initial state of problem

loop do

if frontier is empty **then**

return failure

choose a **leaf node** and remove it from the frontier

if node contains a **goal state** **then**

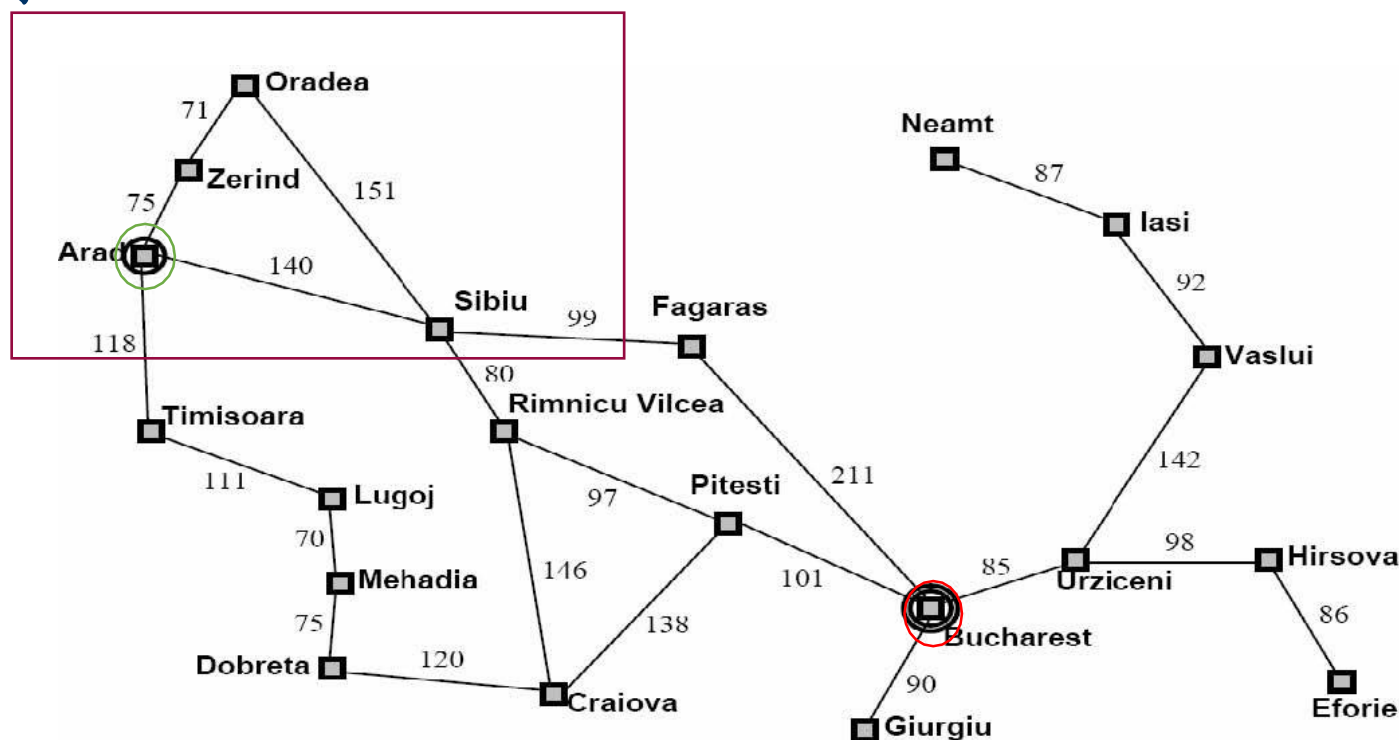
return **solution**(node)

expand the node, adding the resulting nodes **to** the frontier

盲目搜索 (uninformed search) =
不知道目标朝哪边，只能按固定规则把可走的路系统地试一遍。

搜索求解-盲目搜索(树搜索)

冗余路径 (redundant state) : 两个状态之间的迁移路径多于一条

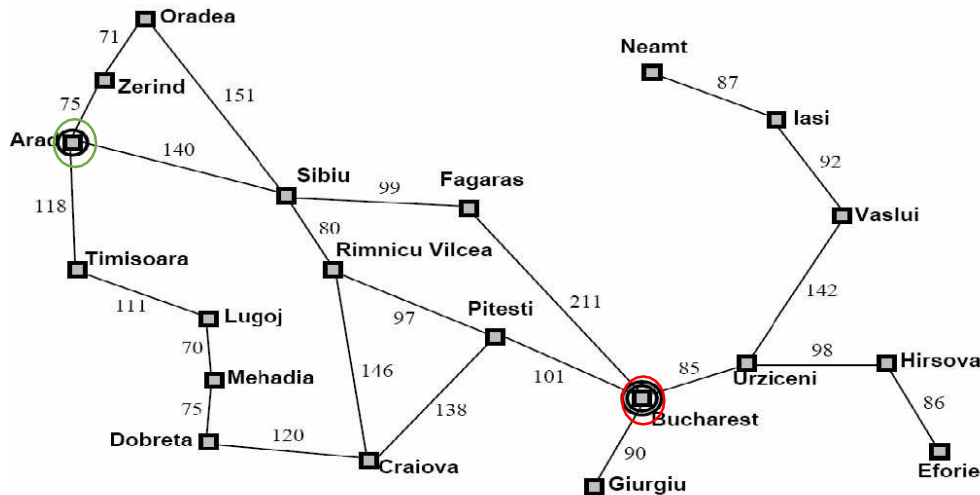


搜索求解-盲目搜索(树搜索)

环路问题:

这种简单的树搜索算法,
因为不检测路径中的重复
城市, 所以可能有**环路**

—— 陷入死循环
(真空吸尘器问题) !



{Arad}

{Z<A>, T<A>, S<A>},

{Z<A>, T<A>, O<S;A>, F<S;A>, A<S;A>, R<S;A>}

{Z<A>, T<A>, O<S;A>, F<S;A>, R<S;A>, Z<A;S;A>, T<A;S;A>, S<A;S;A>}

搜索求解-盲目搜索(图搜索)

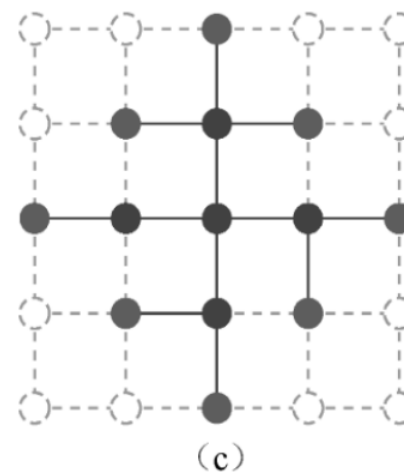
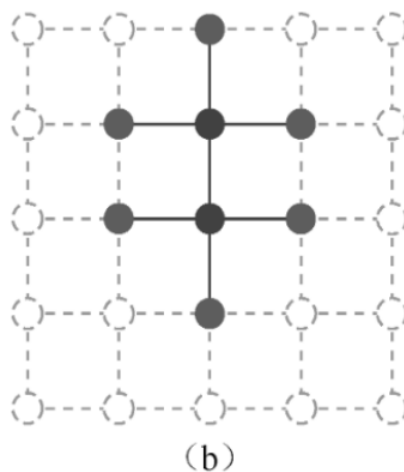
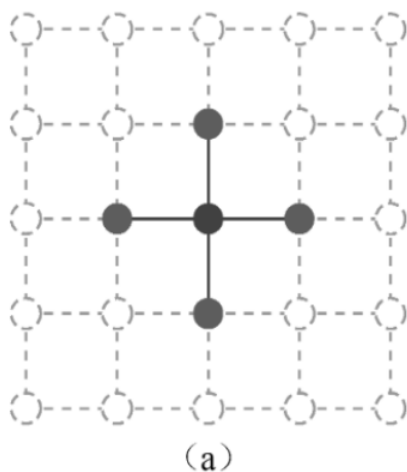
树搜索 (Tree Search)

- 每次扩展节点时，不检查是否已访问过，于是同一个状态可能在搜索树中出现多次。
- 结果：如果图里有环，可能无限扩展；即使没有环，也会浪费时间在重复状态上。

图搜索 (Graph Search)

- 每次扩展节点时，要维护一个**探索集 (closed list)**。
- 新生成的后继状态如果已经访问过，就丢掉，不再扩展。
- 结果：避免环路和重复，效率更高，更接近**现实可用的算法**。

搜索求解-盲目搜索(图搜索)



黑色-已探索区域
浅灰色-未探索区域

搜索求解-盲目搜索(树搜索)

- 从例子中可见，从边缘中选择状态的顺序对于搜索操作的如下性能有很大影响：
 - 是否发现解
 - 解本身的代价
 - 搜索的时空代价

描述搜索算法的性能

- **完备性 (completeness)** : 当问题存在解的时候, 这个算法是否能保证找到最优解; 当不存在解时, 是否能保证报告失败?
- **最优性 (cost optimality)** : 搜索将总是找到成本最低的解决方案吗? (当动作有**成本/路径耗散 (path cost)** 时)
- **时间复杂度 (time complexity)** : 找到解需要花费多少时间?
- **空间复杂度 (space complexity)** : 在执行搜索的过程中需要多少内存 (必须在内存中存储的最大结点数) ?

盲目搜索策略

- 使用固定的后续状态搜索规则
算法一旦选定了一个节点展开，生成它的所有后继状态时，遵循的顺序和方式是固定的（先入先出/后入后出/累计代价最小）
- 规则不改变，不管搜索问题是否解决
（无论目标是不是已经很近，或者是不是已经找到部分线索，搜索过程都严格按原来的规则进行，不会切换策略。）
- 这些策略也不考虑特定搜索问题的领域知识
完全不知道目标在哪里（没有指南针的航行）

盲目搜索的特点：

扩展规则一成不变，不会根据问题的进展调整，也不会利用任何目标相关的知识。它能保证系统性，但效率往往低下。

常见的盲目搜索策略

- 广度优先
- 均一代价
- 深度优先
- 深度限制
- 迭代深入搜索

小结（树搜索）

- 树搜索算法
 - 边缘、路径、问题解、解的代价
- 搜索算法的主要性能评价指标
 - 完备性、最优性、时空复杂度
- 搜索规则影响算法性能
 - 排序、选择、检查